

与 reader_cnt), 因此偏向写者的读写锁提供了一把读者和写者共享的锁 rwlock->lock, 要求读者与写者在操作元数据前获取该锁。

代码片段 10.12 偏向写者的读写锁

```
1 struct rwlock {
2     volatile int reader_cnt;
3     volatile bool has_writer;
4     struct lock lock;
5     struct cond reader_cond;
6     struct cond writer_cond;
7 };
8
9 void lock_reader(struct rwlock *rwlock)
10 {
11     lock(&rwlock->lock);
12     while(rwlock->has_writer == TRUE)
13         cond_wait(&rwlock->writer_cond, &rwlock->lock);
14     rwlock->reader_cnt++;
15     unlock(&rwlock->lock);
16 }
17
18 void unlock_reader(struct rwlock *rwlock)
19 {
20     lock(&rwlock->lock);
21     rwlock->reader_cnt--;
22     if (rwlock->reader_cnt == 0)
23         cond_signal(&rwlock->reader_cond);
24     unlock(&rwlock->lock);
25 }
26
27 void lock_writer(struct rwlock *rwlock)
28 {
29     lock(&rwlock->lock);
30     while(rwlock->has_writer == TRUE)
31         cond_wait(&rwlock->writer_cond, &rwlock->lock);
32     rwlock->has_writer = TRUE;
33     while(rwlock->reader_cnt > 0)
34         cond_wait(&rwlock->reader_cond, &rwlock->lock);
35     unlock(&rwlock->lock);
36 }
37
38 void unlock_writer(struct rwlock *rwlock)
39 {
40     lock(&rwlock->lock);
41     rwlock->has_writer = FALSE;
42     cond_broadcast(&rwlock->writer_cond);
43     unlock(&rwlock->lock);
44 }
```

读者在申请进入临界区 (`lock_reader`) 时, 如果观察到 `has_writer` 被置为 `TRUE` (第 12 行), 则不能进入临界区, 需要使用条件变量挂起等待 (第 13 行)。确保没有写者等待时, 读者在增加读者计数器 `reader_cnt` 并放锁后便可进入读临界区 (第 14、15 行)。

对应地, 在写者需要进入临界区 (`lock_writer`) 时, 也需要通过 `has_writer` 判断是否有其他写者在临界区内 (第 30 行)。如果存在其他写者, 则需要使用条件变量挂起等待 (第 31 行)。当没有其他写者时, 当前写者就可以将 `has_writer` 设置为 `TRUE` (第 32 行), 并使用条件变量等待之前所有的读者离开读临界区 (第 33、34 行)。最后, 释放保护元数据的锁即可进入写临界区 (第 35 行)。

读者在放锁时 (`unlock_reader`), 除了需要减少读者的计数器 `reader_cnt` 值 (第 21 行), 还需要判断自己是不是最后一个读者 (第 22 行)。如果是最后一个读者, 还需要使用 `cond_signal` 来唤醒等待在 `reader_cnt` 上的写者 (第 23 行)。

写者在放锁时 (`unlock_writer`), 除了设置 `has_writer` 为 `FALSE` (第 41 行), 还应当使用 `cond_broadcast` 将等待在 `has_writer` 上的读者和写者都唤醒 (第 42 行)。注意这里使用 `cond_broadcast` 是因为读者与写者都有可能等待在 `has_writer` 上, 因此两者都需要被唤醒。如果使用 `cond_signal`, 有可能只唤醒了一个读者, 那么等待在 `has_writer` 上的写者将永远不会被唤醒, 从而陷入无限等待。

从上面介绍的两种读写锁的实现中我们可以发现, 偏向读者的读写锁能大幅提高读者之间的并行度, 但是写者由于需要等待某一个时刻完全没有读者时才能更新数据, 需要面临很高的写延迟。而偏向写者的读写锁虽然避免了这个问题, 让写者能够在之前的读者完成读临界区后立刻进入, 但读者之间的并行度大幅下降。因此开发者需要根据具体场景的需求选用合适的读写锁。

10.5 案例分析: Linux 中的 `futex`

为了解决循环忙等的问题, 我们在 10.2 节介绍了条件变量。但条件变量需要与一个基于循环等待的互斥锁搭配使用。如果需要在互斥锁 (比如代码片段 10.4 中实现的自旋锁) 中使用条件变量避免循环, 那么在每次上锁时需要额外进行加锁 / 放锁操作, 这会带来不必要的开销。这种性能开销在互斥锁竞争程度低时更为显著: 之前的实现只需要一次原子操作即可上锁, 而使用了条件变量后则需要两次原子操作 (额外的用于保护条件变量的互斥锁)。另一个方案是直接使用如 `SYS_yield` 的系统调用来挂起自己。但这样挂起与唤醒时机完全是由操作系统调度器控制的, 可能出现还没挂起时锁已经释放或锁释放很久后线程才被唤醒的情况。

因此, Linux 提供了 `futex` (Fast User-space muTEX) 机制。使用 `futex` 机制实现的互斥锁能在竞争程度较低时, 直接使用原子操作完成加锁。而竞争程度较高时, 应用程序能够通过系统调用挂起并等待后续锁持有者被唤醒。`futex` 机制不仅可以避免互斥锁中的

循环等待，还能实现条件变量等各类同步原语。

Linux 内核中的 `futex` 功能完善且繁多，本小节将介绍简化后的 `futex` 接口 `futex_wait` 和 `futex_wake`，分别用于等待和唤醒。其中 `futex_wait` 接收两个参数：`uaddr` 和 `val`。`futex_wait` 操作会判断地址 `uaddr` 上的值是否与 `val` 相等。如果相等则挂起该线程，否则直接返回。`futex_wake` 只接收一个参数 `uaddr`，其会尝试唤醒之前通过 `futex_wait` 等待在相同 `uaddr` 上的线程。这里我们简化的接口只会唤醒一个线程，而 Linux 中的 `futex` 可以通过配置参数唤醒指定数量的线程。`futex` 提供的这两个操作是互斥（即线程安全）的。

代码片段 10.13 展示了简化的 `futex` 的实现。需要注意的是，这里仅展示了 `futex` 的功能与原理，并非 Linux 中的实现。对 Linux 中 `futex` 的实现感兴趣的读者可以自行阅读 Linux 源码。针对每一个 `futex`，内核中维护了一个 `futex` 结构体，用于保存维护 `futex` 操作互斥性的互斥锁 `lock`，以及等待在该 `futex` 上的等待队列 `wait_list`。内核提供了一个简单的函数 `get_futex`，用于将 `futex` 的地址映射到该结构体上，内核可以使用哈希表的方式存储该结构体。`futex_wait` 与 `futex_wake` 操作均使用互斥锁保护起来，在开始与结束时分别有加锁与放锁操作。`futex_wait` 在成功获取互斥锁后才会判断 `uaddr` 中存储的值与 `val` 是否相等（第 12 行），并在相等时通过设置当前线程状态（第 13 行）且不直接返回用户态来挂起当前线程。`futex_wait` 还需要将自己加入当前 `futex` 的等待队列（第 14 行），并在放锁后将控制流移交给调度器，让调度器选择下一个执行的线程并切换。当 `uaddr` 中存储的值与 `val` 不相等时，代表在这段时间内有其他线程调用了 `futex_wake` 操作，此时其会直接解锁并返回当前线程的用户态（第 19 行）。由于 `futex_wait` 在内核态中，因此要实现阻塞当前线程，仅仅需要更新当前线程状态、不将其加入调度器的等待队列并要求调度器选择其他线程调度。在完成这些操作之后再释放互斥锁即可实现原子阻塞与放锁，从而避免丢失唤醒。我们将在下面通过互斥锁实现的例子详细解释为何不会出现丢失唤醒。与之相对，`futex_wake` 操作会在成功获取互斥锁之后判断是否有线程等待在当前的 `futex` 上（第 30 行）。一旦发现有等待的线程，其会将该线程移出等待队列（第 31 行），设置该线程的状态（第 32 行），将其加入调度器的等待队列，并允许调度器调度该线程执行。

代码片段 10.13 简化的 `futex` 实现

```
1 struct futex {
2     struct thread_list *wait_list;
3     struct lock lock;
4 };
5 struct thread *current_thread; // 记录当前运行的线程
6
7 void futex_wait(int *uaddr, int val)
8 {
9     struct futex *fptr = get_futex(uaddr);
```

```

10
11  lock(&fptr->lock);
12  if (*uaddr == val) {
13      current_thread->state = WAITING;
14      list_append(fp->wait_list, current_thread);
15      unlock(&fptr->lock);
16      sched_and_ret();
17      // 调度器选择下一个线程并切换到该线程，不会返回
18  } else {
19      unlock(&fptr->lock);
20      // 直接返回当前线程用户态
21  }
22 }
23
24 void futex_wake(int *uaddr)
25 {
26     struct futex *fptr = get_futex(uaddr);
27     struct thread *target;
28
29     lock(&fptr->lock);
30     if (!list_empty(fp->wait_list)) {
31         target = list_remove(fp->wait_list);
32         target->state=READY;
33         sched_enqueue(target);
34     }
35     unlock(&fptr->lock);
36     // 直接返回当前线程用户态
37 }

```

代码片段 10.14 利用简化版的 futex 接口实现 lock 和 unlock 功能。在 lock 操作尝试 atomic_CAS 失败后（第 8 行），会先使用原子操作增加等待者计数器 waiters（第 9 行），然后调用 futex_wait 进入内核尝试挂起（第 10 行）。由于用户态 atomic_CAS 执行和最终挂起之间存在时间差，在这段时间内锁可能已经被释放，因此内核中会检查传入的 uaddr（即 lock->val）上的值是否为 val（即 1）。如果不相等，则代表 lock->val 的值为 0，说明锁已经被释放，此时会直接返回用户态，并再次尝试 atomic_CAS；相反，如果 lock->val 的值为 1，则说明该线程需要挂起。等待者计数器 waiters 记录了当前有多少线程调用 futex_wait，如果在放锁时 waiters 的值不为 0（第 18 行），则使用 futex_wake 尝试唤醒正在等待加锁的线程（第 19 行）。

代码片段 10.14 使用简化的 futex 接口实现的互斥锁

```

1 struct lock {
2     int val;
3     int waiters;
4 };
5
6 void lock(struct lock *lock)

```



```
7 {
8   while(atomic_CAS(&lock->val, 0, 1) != 0) {
9       atomic_FAA(&lock->waiters, 1);
10      futex_wait(&lock->val, 1);
11      atomic_FAA(&lock->waiters, -1);
12  }
13 }
14
15 void unlock(struct lock *lock)
16 {
17     lock->val = 0;
18     if (lock->waiters != 0)
19         futex_wake(&lock->val);
20 }
```

下面我们讨论在使用 futex 实现的互斥锁中是否存在丢失唤醒的现象。比如有线程在调用 futex_wait，但还未将线程插入内核的等待队列时，另一个线程同时调用 futex_wake，发现等待队列为空，就有可能造成丢失唤醒。为了避免这种情况，不同 futex 操作均使用互斥锁进行保护（如代码片段 10.13 中所示），因此多个 futex 操作之间只可能先后发生，不可能同时发生。因此在上面这种情况中：

1. 如果 futex_wait 操作先发生（代码片段 10.14 的第 10 行），即成功进入内核并拿到该 futex 的互斥锁（代码片段 10.13 的第 11 行），那么内核能保证该线程在另一个线程调用 futex_wake 并拿到互斥锁之前：

（a）要么回到用户态（lock->val 已经更新为 0，即另一个线程执行了代码片段 10.14 的第 17 行）。此时 futex_wait 操作发现值更改后直接放锁并返回（代码片段 10.13 中的第 19 行）。

（b）要么进入等待队列（lock->val 还未更新）。此时 futex_wait 先将当前线程加入等待队列中再释放互斥锁（代码片段 10.13 的第 14 行）。之后 futex_wake 获取互斥锁时会发现该线程（代码片段 10.13 的第 30 行）并唤醒。

2. 如果 futex_wake（代码片段 10.14 的第 19 行）操作先发生，即其先拿到 futex 的互斥锁（代码片段 10.13 的第 29 行），则 lock->val 一定已经更新为 0，因此后续的 futex_wait 在拿到互斥锁后会立即返回用户态。

所以在任何情况下，调用 futex_wait 的线程都不会由于错过 futex_wake 而陷入无限等待并造成丢失唤醒。

代码片段 10.15 展示了如何使用 futex 实现条件变量的两个操作。这里提供了简化的实现，其中 cond_signal 只能在与之搭配锁的临界区内被调用。条件变量提供了一个成员变量 value 作为 futex 操作的对象。cond_wait 操作将先读取 value 的旧值并将其存到局部变量 local 中（第 7 行），然后释放互斥锁（第 8 行），之后利用 futex_wait 操作挂起（第 9 行）。唤醒后，该线程需要重新获取互斥锁（第 10 行）。而 cond_

signal 操作则先更新 value 的值，然后调用 `futex_wake` 唤醒挂起的线程。该实现也不存在丢失唤醒的情况：如果执行 `cond_wait` 操作的线程在放锁之后（第 8 行）还未执行 `futex_wait`（第 9 行）时，有其他线程进入临界区并调用了 `futex_wake`（第 16 行），那么条件变量的 value 也一定被更新（第 15 行），因此 `futex_wait` 会发现该值与本地存储的旧值（`local`）不相等，将立即返回。否则，该线程将挂起并等待后续线程唤醒。因此，这里将条件变量“原子放锁并挂起”的需求转嫁到一个 `futex` 上，通过内核的支持正确实现了条件变量的语义。

代码片段 10.15 简化的 `futex` 接口和使用 `futex` 的条件变量

```
1 struct cond {
2     unsigned value;
3 };
4
5 void cond_wait(struct cond *cond, struct lock *mutex)
6 {
7     unsigned local = cond->value;
8     unlock(mutex);
9     futex_wait(&cond->value, local);
10    lock(mutex);
11 }
12
13 void cond_signal(struct cond *cond)
14 {
15     cond->value += 1;
16     futex_wake(&cond->value);
17 }
```

小思考

代码片段 10.15 中利用 `futex` 实现条件变量会有什么问题？

需要注意的是，代码片段 10.15 中由于 `value` 使用的类型为 `unsigned`，因此可能会发生溢出。如果 `cond_wait` 的线程放锁之后，`cond_signal` 被调用了 2^{32} 次，则 `value` 的值会溢出，导致调用 `cond_wait` 的线程丢失信号并陷入无限等待。因此，真实场景中将使用更加复杂的设计以避免溢出问题。

10.6 案例分析：微内核中的同步原语

微内核中同步原语的使用与实现与宏内核基本一致。我们可以将同步原语分为两类。第一类同步原语的实现与使用均与操作系统内核无关，如本章介绍的互斥锁和读写锁。这些同步原语在宏内核与微内核中的实现完全一致，并且可以在内核态与用户态直